

यूनिट 1 एल्गोरिदम की मूल बातें और इसके गुण

1.0 परिचय 1.1 उद्देश्य 1.2

एल्गोरिदम का उदाहरण

1.3 बेसिक्स एल्गोरिदम के बिल्डिंग ब्लॉक्स

1.3.1 सीक्वेंसिंग सिलेक्शन और इटरेशन 1.3.2 प्रोसीजर और रिकर्सन

1.4 कॉमन रनिंग टाइम का सर्वे 1.5 एल्गोरिदम का एनालिसिस और

कॉम्प्लेक्सिटी 1.6 प्रॉब्लम के टाइप 1.7 प्रॉब्लम सॉल्विंग टेक्नीक 1.8

डिटरमिनिस्टिक और स्टोकेस्टिक

एल्गोरिदम 1.9 समरी 1.10 चैप्टर रिव्यू सवाल 1.11 आगे

की रीडिंग

1.0 परिचय

कंप्यूटर साइंस में एल्गोरिदम की पढ़ाई एक रोमांचक विषय है। हमें इन समस्याओं को हल करने के लिए बहुत सारी दिलचस्प समस्याएं और तकनीकें मिलती हैं। हर समस्या को मौजूदा तकनीकों से हल नहीं किया जा सकता, लेकिन उनमें से ज्यादातर को किया जा सकता है। लेकिन आइए पहले यह समझते हैं कि एल्गोरिदम क्या है। एल्गोरिदम शब्द नौवीं सदी के गणितज्ञ अब्दुल्ला जफर मुहम्मद इब्न मूसा अल-खोवारिज्मी से लिया गया है। 'अल-खोवारिज्मी' शब्द लैटिन में 'एल्गोरिज्मस' है जो उनके नाम पर एल्गोरिदम बन गया।

उन्होंने एल्गोरिदम को इस तरह बताया:

- एल्गोरिदम हाथ से कैलकुलेशन करने के लिए नियमों का एक सेट है या किसी मशीन पर।
- एक एल्गोरिदम एक अच्छी तरह से परिभाषित कम्प्यूटेशनल प्रक्रिया है जो इनपुट और आउटपुट बनाता है।
- एक एल्गोरिदम कुछ खास आउटपुट पाने के लिए इंस्ट्रक्शन या स्टेप्स (यानी इनपुट) का एक सीमित सीक्वेंस है।

इस यूनिट में, एल्गोरिदम की बेसिक बातें और इसके डिजाइनिंग प्रोसेस पर चर्चा की जाएगी। सेक्शन 1.3 में एल्गोरिदम और इसके इस्तेमाल को सही उदाहरण के साथ बताया जाएगा। एक एल्गोरिदम को पांच बेसिक बिल्डिंग ब्लॉक्स के साथ डिजाइन किया जाता है, यानी सीक्वेंसिंग, सिलेक्शन और इटरेशन। एल्गोरिदम के इन बिल्डिंग ब्लॉक्स के बारे में डिटेल् में चर्चा सेक्शन 1.4 में दी गई है।

एल्गोरिदम की मूल बातें और
इसके गुण

किसी प्रॉब्लम का सॉल्यूशन कई एल्गोरिदम से किया जा सकता है। यह चेक करने के लिए कि कौन सा एल्गोरिदम दूसरों से बेहतर है, टाइम कॉम्प्लेक्सिटी नाम का एक पैरामीटर इस्तेमाल किया जाता है। इसलिए, टाइम कॉम्प्लेक्सिटी एल्गोरिदम से जुड़े ज़रूरी कॉन्सेप्ट में से एक है, जिस पर सेक्शन 1.5 में बात की गई है। सेक्शन 1.6 एल्गोरिदम के एनालिसिस से जुड़ा है। एल्गोरिदम की तुलना करने के लिए, कॉम्प्लेक्सिटी वह पैरामीटर है जिस पर ध्यान देना है। कंप्यूटिंग प्रॉब्लम को उनके सॉल्विंग अप्रोच के हिसाब से कैटेगरीज़ किया जाता है। इन पर सेक्शन 1.7 में बात की गई है। सेक्शन 1.8 में अलग-अलग कंप्यूटिंग प्रॉब्लम को सॉल्व करने की टेक्नीक शामिल हैं। सेक्शन 1.9 में डिटरमिनिस्टिक और स्टोकेस्टिक एल्गोरिदम पर बात की गई है। एक एल्गोरिदम डिटरमिनिस्टिक होता है अगर अगले आउटपुट को इनपुट और प्रोग्राम के स्टेट से प्रेडिक्ट/डिटर किया जा सके, जबकि स्टोकेस्टिक एल्गोरिदम रैंडम होते हैं।

प्रकृति।

चैप्टर को सेक्शन 1.10 में समराइज़ किया गया है। सेक्शन 1.11 में चैप्टर के रिव्यू सवाल चेक पॉइंटिंग के मकसद से कवर किए गए हैं। सेक्शन 1.12 में आगे पढ़ने के लिए रेफरेंस मटीरियल की एक लिस्ट दी गई है।

1.1 उद्देश्य

इस यूनिट को पढ़ने के बाद, आप ये कर पाएंगे:

- एल्गोरिदम की प्रॉपर्टीज़ को डिफाइन करें और लिस्ट करें। •
 - एल्गोरिदम के बेसिक बिल्डिंग ब्लॉक्स को लिस्ट करें। •
 - एल्गोरिदम डिज़ाइन करने की बेसिक टेक्नीक्स समझाएं। • एल्गोरिदम की टाइम और स्पेस कॉम्प्लेक्सिटी को डिफाइन करें। • डिटरमिनिस्टिक और स्टोकेस्टिक एल्गोरिदम के बीच अंतर बताएं।
-

1.2 एल्गोरिदम का उदाहरण

एल्गोरिदम कोई कोडिंग इंस्ट्रक्शन नहीं है, बल्कि यह आम भाषा में लिखे गए कामों का एक सीक्वेंस है, जिसे अगर पूरा किया जाए तो एक टाइम फ्रेम में कुछ आउटपुट मिलता है। एक एल्गोरिदम प्रोग्रामिंग भाषा से पूरी तरह से स्वतंत्र होता है।

एक अच्छे एल्गोरिदम के लिए, उसे ये बातें पूरी करनी होंगी:

1. इनपुट: एल्गोरिदम के लिए इनपुट की एक सीमित संख्या होनी चाहिए।
2. आउटपुट: एल्गोरिदम के एग्जीक्यूशन के नतीजे में कुछ आउटपुट ज़रूर निकलना चाहिए।
3. निश्चितता: इनपुट को आउटपुट में बदलने के लिए ऑपरेशन का एक निश्चित क्रम होना चाहिए।
4. असर: एल्गोरिदम का हर स्टेप बेसिक और आवश्यक।

5. सीमितता: इनपुट से आउटपुट में बदलाव सीमित स्टेप्स में होना चाहिए, यानी एल्गोरिदम को आखिरकार रुकना ही होगा! रुकने का मतलब यह हो सकता है कि यह उम्मीद के मुताबिक आउटपुट दे या ऐसा जवाब दे कि कोई हल मुमकिन नहीं है।

एक एल्गोरिदम की ज़रूरी खासियतें ये हैं:

- एल्गोरिदम आम होना चाहिए और कई मामलों को हल कर सके। • एल्गोरिदम को रिसोर्स का अच्छे से इस्तेमाल करना चाहिए, यानी कम समय लेना चाहिए।
और रिज़ल्ट बनाने में मेमोरी।
- एल्गोरिदम समझने लायक होने चाहिए ताकि कोई भी उन्हें समझ सके
इसे समझें और अपनी समस्या पर लागू करें।
- एल्गोरिदम को यूनिक्नेस को फॉलो करना चाहिए ताकि एल्गोरिदम का हर इंस्ट्रक्शन साफ़ और साफ़ हो।

किसी एल्गोरिदम का “एनालिसिस” समय और जगह (मेमोरी) की जटिलता के हिसाब से उसकी सही होने की जांच करता है, जिसे एल्गोरिदम का परफॉर्मेंस इवैल्यूएशन भी कहते हैं।

एल्गोरिदम लिखने और उसके एनालिसिस के बारे में आगे बात करने से पहले, आइए पहले देखते हैं कि एल्गोरिदम कैसा दिखता है। इसके लिए, आइए दो दिए गए इंटीजर का ग्रेटेस्ट कॉमन डिवाइज़र (GCD) निकालने के लिए एक जाने-माने एल्गोरिदम पर विचार करें। हम किसी भी नेचुरल लैंग्वेज में एल्गोरिदम लिख सकते हैं।

यहां, हम एल्गोरिदम को अंग्रेजी भाषा में प्रस्तुत कर रहे हैं।

दो दिए गए इंटीजर का GCD पता करने के लिए, हम एक कुशल यूक्लिड एल्गोरिदम का इस्तेमाल कर रहे हैं जिसका नाम प्राचीन ग्रीक गणितज्ञ यूक्लिड के नाम पर रखा गया है।

यूक्लिड मेथड से GCD (a, b) कैलकुलेट करने के लिए स्यूडो कोड इस तरह है:

// a और b दो पॉज़िटिव नंबर हैं जहाँ a डिविडेंड है और b डिवाइज़र है

1. अगर $b=0$ है, तो a को वापस करें और बाहर निकलें
2. नहीं तो स्टेप 3
- पर जाएं 3. a को b से डिवाइड करें और बाकी r को असाइन करें
4. b की वैल्यू a को और r की वैल्यू b को असाइन करें और स्टेप 1 पर वापस जाएं

एल्गोरिदम को वैलिडेट करने के लिए, उसे कुछ ही स्टेप्स में मनचाहा रिज़ल्ट देना होगा। ऊपर बताए गए एल्गोरिदम में दो इनपुट और एक आउटपुट है। एल्गोरिदम भी निश्चित है और बेसिक और असरदार वाक्यों में लिखा गया है। एल्गोरिदम भी सीमित है क्योंकि यह सीमित स्टेप्स में खत्म होता है। इसे देखने के लिए, आइए यूक्लिड के एल्गोरिदम का इस्तेमाल करके $a = 1071$ और $b = 462$ का GCD पता करें।

पुनरावृत्ति 1:

1. $a=1071$ को $b=462$ से भाग दें और शेष को r में जमा करें।

आर = 1071 % 462

(यहां, % शेष ऑपरेटर को दर्शाता है)

एल्गोरिदम की मूल बातें और
इसके गुण

आर = 147

2. अगर $r = 0$, तो एल्गोरिदम खत्म हो जाता है और b GCD है। नहीं तो, यहाँ जाएँ
स्टेप 3.

यहां, r ज़ीरो नहीं है, इसलिए हम स्टेप 3 पर जाएंगे।

3. पूर्णांक को पूर्णांक b का वर्तमान मान और का नया मान मिलेगा
पूर्णांक b , r का वर्तमान मान होगा।

यहाँ, $a=462$ और $b=147$

4. स्टेप 1 पर वापस जाएं।

पुनरावृत्ति 2:

1. $a = 462$ को $b = 147$ से भाग दें और शेष को r में जमा करें।

$r = 462 \% 147$ (यहां, $\%$ शेष ऑपरेटर को दिखाता है)

आर = 21

2. अगर $r = 0$, तो एल्गोरिदम खत्म हो जाता है और b GCD है। नहीं तो, यहाँ जाएँ
स्टेप 3.

यहां, r ज़ीरो नहीं है, इसलिए हम स्टेप 3 पर जाएंगे।

3. पूर्णांक a को पूर्णांक b का वर्तमान मान और नया मान मिलेगा
पूर्णांक b का मान r का वर्तमान मान होगा।

यहाँ, $a = 147$ और $b = 21$

4. स्टेप 1 पर वापस जाएं।

पुनरावृत्ति 3:

1. $a = 147$ को $b = 21$ से भाग दें और शेष को r में जमा करें।

$r = 147 \% 21$ (यहां, $\%$ शेष ऑपरेटर को दिखाता है)

आर = 0

2. अगर $r = 0$, तो एल्गोरिदम खत्म हो जाता है और b GCD है। नहीं तो, यहाँ जाएँ
स्टेप 3.

यहां, r ज़ीरो है, इसलिए एल्गोरिदम खत्म हो जाता है और b जवाब है, यानी 21.

आइए यूक्लिड का एल्गोरिदम लिखें

```

एल्गोरिथम GCD-यूक्लिड (a, b)
आरंभ [एल्गोरिदम की शुरुआत]
{
    जबकि b  $\neq$  0
    {
        आर  $\leftarrow$  ए मॉड बी;
        ए  $\leftarrow$  बी;
        बी  $\leftarrow$  आर;
    } [while लूप का अंत]
वापसी (बी)
} [एल्गोरिदम का अंत]

```

एल्गोरिदम 1: यूक्लिडियन विधि से (a, b) का GCD ज्ञात करना।

1.3 बुनियादी बातें

एल्गोरिदम

एल्गोरिदम किसी प्रॉब्लम का सॉल्यूशन लिखने का एक प्रोसिजरल तरीका है। इसे पाँच बेसिक बिल्डिंग ब्लॉक्स के साथ डिज़ाइन किया गया है, यानी: सीक्वेंसिंग, सिलेक्शन, इटरेशन प्रोसिजर और रिकर्सन। पहले सब-सेक्शन में हम सीक्वेंसिंग, इटरेशन और सिलेक्शन पर बात करेंगे और अगले सब-सेक्शन में प्रोसिजर और रिकर्सन के बारे में बताया जाएगा।

सीनियर एन.	बिल्डिंग ब्लॉक	साधारण नाम
1.	अनुक्रमण	चरण दर चरण कार्रवाई
2.	चयन	फ़ैसला
3.	यात्रा	पुनरावृत्ति या लूप
4.	प्रक्रिया	
5.	प्रत्यावर्तन	

1.3.1 अनुक्रमण, चयन और पुनरावृत्ति

किसी प्रॉब्लम का सॉल्यूशन मुख्य रूप से इन तीन बेसिक ब्लॉक से ही बनता है। एक Algorithm में टेक्स्ट में लिखे अनुसार लीनियरली या सीक्वेंस में किए जाने वाले एक्शन हो सकते हैं। कभी-कभी अगला एक्शन/स्टेटमेंट जो किया जाना है वह है

एल्गोरिदम की मूल बातें और
इसके गुण

यह कुछ कंडीशन के आधार पर तय होता है, जिसे अगले एक्शन का सिलेक्शन कहते हैं। कुछ एक्शन/स्टेटमेंट को एक से ज़्यादा बार एग्जीक्यूट करना होता है, जिसे रिपीटिशन या लूप कहते हैं।

आइए, एक एल्गोरिदम के बेसिक बिल्डिंग ब्लॉक्स को समझने के लिए यूक्लिडियन मेथड से (a, b) का GCD निकालने के उदाहरण पर विचार करें (एल्गोरिदम 1)

सीक्वेंसिंग: किसी प्रॉब्लम को कुछ एक्शन को एक सीक्वेंस में करके [जिसे एल्गोरिदम कहते हैं] हल किया जा सकता है, और उन एक्शन को करने का क्रम एल्गोरिदम के सही होने को पक्का करने के लिए ज़रूरी है।

अगर एल्गोरिदम के स्टेप्स का क्रम बदल जाता है और बताए गए स्टेप्स को फॉलो नहीं करता है, तो यह उम्मीद के मुताबिक सही आउटपुट नहीं देगा।

सिलेक्शन: क्योंकि कई मामलों को हल करने के लिए एक एल्गोरिदम को जनरलाइज़ करना पड़ता है, ऐसे हालात हो सकते हैं जहाँ एक्शन के एग्जीक्यूशन का सीक्वेंस किसी कंडीशन पर निर्भर हो सकता है। यानी, कुछ इंडस्ट्रक्शन तभी एग्जीक्यूट होंगे जब दी गई कंडीशन पूरी होगी।

यह ऐसा है जैसे किया जाने वाला अगला एक्शन किसी बूलियन एक्सप्रेशन पर निर्भर करता है। इसलिए, सिलेक्शन का इस्तेमाल करके, किया जाने वाला अगला स्टेप तय किया जाता है।

इटरेशन: किसी प्रॉब्लम को सॉल्व करते समय कुछ एक्शन को एक तय संख्या में या एक तय कंडीशन पूरी होने तक एग्जीक्यूट करने की ज़रूरत हो सकती है।

इन कॉन्सेप्ट्स को समझने के लिए आइए एल्गोरिदम 1 पर फिर से विचार करें: यूक्लिडियन मेथड से (a, b) का GCD निकालना।

एल्गोरिथ्म GCD-यूक्लिड (a, b)

स्टेप 1: शुरू करें [एल्गोरिदम की शुरुआत]

चरण दो: {

स्टेप 3: जब $b \neq 0$ हो, तो

चरण 4: {

चरण 5: $r \leftarrow a \bmod b$;

चरण 6: $a \leftarrow b$;

चरण 7: $b \leftarrow r$;

स्टेप 8: } [while लूप का अंत]

चरण 9: वापसी (b)

स्टेप 10: } [एल्गोरिदम का अंत]

ऊपर दिए गए एल्गोरिदम में Step5, Step6 और Step7 सीक्वेंसिंग के उदाहरण हैं, क्योंकि ये स्टेटमेंट हमेशा लिखे गए टेक्स्ट के सीक्वेंस में ही एग्जीक्यूट होते हैं।

स्टेप 3 सिलेक्शन स्टेप है क्योंकि यह तय करता है कि लूप की कंडीशन के अनुसार स्टेप 4 और स्टेप 9 में से अगला कौन सा स्टेप एग्जीक्यूट किया जाना है।

स्टेप 3 इटरेशन या लूपिंग स्टेटमेंट के तौर पर भी काम करता है। while लूप कंडीशन के आधार पर, स्टेप 4 से स्टेप 8 को बार-बार एग्जीक्यूट किया जाता है।

1.3.2 प्रक्रिया और पुनरावृत्ति

यद्यपि उपर्युक्त तीन नियंत्रण संरचनाएं, अर्थात प्रत्यक्ष अनुक्रमण, चयन और पुनरावृत्ति किसी भी एल्गोरिदम को व्यक्त करने के लिए पर्याप्त हैं, फिर भी निम्नलिखित दो उन्नत नियंत्रण संरचनाएं जटिल एल्गोरिदम की अभिव्यक्ति को सुविधाजनक बनाने में काफी उपयोगी साबित हुई हैं।

(i) प्रक्रिया

(ii) पुनरावृत्ति

आइए सबसे पहले एडवांस्ड कंट्रोल स्ट्रक्चर प्रोसीजर को समझते हैं।

प्रक्रिया

प्रोसीजर की जगह इस्तेमाल होने वाले कई शब्दों में सबप्रोग्राम और फंक्शन भी शामिल हैं। अलग-अलग प्रोग्रामिंग लैंग्वेज में इन शब्दों के इस्तेमाल में थोड़ा अंतर हो सकता है। हालांकि, इन शब्दों के पीछे का बेसिक आइडिया एक ही है, और इसे आगे समझाया गया है।

ऐसा हो सकता है कि कोई सीक्वेंस बार-बार या तो एक ही एल्गोरिदम में एल्गोरिदम के अलग-अलग हिस्सों में बार-बार आए या अलग-अलग एल्गोरिदम में हो। ऐसे मामलों में, एक ही सीक्वेंस को बार-बार लिखना एक बेकार काम है। प्रोसीजर एक ऐसा मैकेनिज्म है जो इस बर्बादी को रोकने का एक तरीका देता है। उदाहरण के लिए, हम $GCD(a, b)$ को सिर्फ एक बार प्रोसीजर/फंक्शन के तौर पर डिफाइन कर सकते हैं और इसे a और b की अलग-अलग वैल्यू वाले मेन फंक्शन में कई बार कॉल कर सकते हैं।

इस मैकेनिज्म के तहत, एक या ज़्यादा एल्गोरिदम में बार-बार इस्तेमाल होने वाले इंस्ट्रक्शन का सीक्वेंस सिर्फ एक बार लिखा जाता है और उन एल्गोरिदम से अलग और इंडिपेंडेंट होता है जिनका वह सीक्वेंस वरना हिस्सा हो सकता था। ऐसे कई सीक्वेंस हो सकते हैं और इसलिए, ऐसे हर सीक्वेंस की पहचान की ज़रूरत होती है। इस मकसद के लिए, हर सीक्वेंस के पहले इस फॉर्मेट में स्टेटमेंट दिए गए हैं:

प्रक्रिया <नाम> (<पैरामीटर-सूची>) [: <प्रकार>]
<घोषणाएँ>
<निर्देशों का क्रम जिसके बार-बार होने की उम्मीद है>

अंत;

ऐसे प्रोसीजर के मामलों में जो कॉलिंग प्रोग्राम को वैल्यू पास करते हैं, एक और बेसिक कंस्ट्रक्ट (असाइनमेंट, रीड और राइट के अलावा) यानी रिटर्न (x) का इस्तेमाल किया जाता है, जहाँ x एक वेरिएबल है जिसका इस्तेमाल प्रोसीजर द्वारा पास की जाने वाली वैल्यू के लिए किया जाता है।

ऐसे कई तरीके हैं जिनसे a और b की वैल्यू को एक के बाद एक x और y से जोड़ा जाता है या उनमें ट्रांसफर किया जाता है। a और b जैसे वेरिएबल, जो कॉलिंग एल्गोरिदम में प्रोसीजर (यानी, कॉल्ड एल्गोरिदम) को डेटा पास करने के लिए डिफाइन किए गए हैं, जिनका इस्तेमाल प्रोसीजर प्रॉब्लम के खास उदाहरण को सॉल्व करने में कर सकता है, उन्हें एक्जुअल पैरामीटर या आर्गुमेंट कहा जाता है।

प्रत्यावर्तन

इसके बाद, हम एक और ज़रूरी कंट्रोल स्ट्रक्चर यानी रिकर्सन पर विचार करेंगे। चर्चा को आसान बनाने के लिए, हम मैथ्स से एक तरीका याद करते हैं,

एल्गोरिदम की मूल बातें और इसके गुण

जिसमें एक प्राकृतिक संख्या n का फैक्टोरियल परिभाषित किया गया है:

$$\text{फैक्टोरियल}(1) = 1$$

$$\text{फैक्टोरियल}(n) = n * \text{फैक्टोरियल}(n-1)$$

जो लोग फैक्टोरियल के लिए ऊपर दी गई पुनरावर्ती परिभाषाओं से परिचित हैं, उनके लिए यह समझना आसान है कि प्राकृतिक संख्या के फैक्टोरियल की उपरोक्त परिभाषा से $(n!)$ का मान कैसे प्राप्त किया जाता है। हालाँकि, जो लोग पुनरावर्ती परिभाषाओं से परिचित नहीं हैं, उनके लिए हम उपरोक्त परिभाषा का उपयोग करके फैक्टोरियल (4) की गणना करेंगे।

परिभाषा के अनुसार

$$\text{फैक्टोरियल}(4) = 4 * \text{फैक्टोरियल}(3)$$

पुनः परिभाषा के अनुसार फैक्टोरियल (3)

$$= 3 * \text{फैक्टोरियल}(2)$$

इसी प्रकार

$$\text{फैक्टोरियल}(2) = 2 * \text{फैक्टोरियल}(1)$$

और परिभाषा के अनुसार

$$\text{फैक्टोरियल}(1) = 1$$

फैक्टोरियल (1) , फैक्टोरियल (2) इत्यादि के मानों को प्रतिस्थापित करने पर, हमें वांछित फैक्टोरियल $(4) = 4.3.2.1=24$ प्राप्त होता है।

यह परिभाषा एक नेचुरल नंबर n का फैक्टोरियल कैलकुलेट करने के लिए नीचे दिया गया प्रोसेस/एल्गोरिदम बताती है:

नीचे दिए गए प्रोसीजर फैक्टोरियल (n) में, फैक्ट को वह वैरिएबल मान लें जिसका इस्तेमाल प्रोसीजर फैक्टोरियल द्वारा वैल्यू को कॉलिंग प्रोग्राम में पास करने के लिए किया जाता है। वैरिएबल फैक्ट को शुरू में वैल्यू 1 दी जाती है, जो फैक्टोरियल (1) की वैल्यू है।

प्रक्रिया फैक्टोरियल (n)

तथ्य: पूर्णांक;

शुरू

तथ्य $\leftarrow 1$

अगर n बराबर 1 है तो फैक्ट लौटाएं

अन्यथा शुरू करें

तथ्य $\leftarrow n * \text{फैक्टोरियल}(n-1)$

वापसी (तथ्य)

अंत;

अंत;

फैक्टोरियल $(n-1)$ कैलकुलेट करने के लिए, प्रोसीजर फैक्टोरियल को खुद ही कॉल किया जाता है, लेकिन इस बार (आसान) आर्गुमेंट $(n-1)$ के साथ। आसान आर्गुमेंट के साथ बार-बार कॉल तब तक जारी रहती है जब तक फैक्टोरियल को आर्गुमेंट 1 के साथ कॉल नहीं किया जाता। पार्शियल रिजल्ट्स को $2,3, \dots, n$ तक के साथ लगातार मल्टीप्लाई करने पर आखिर में मनचाहा रिजल्ट मिलता है।

परिभाषा: एक प्रोसीजर, जो खुद को कॉल कर सकता है, उसे रिकर्सिव प्रोसीजर/एल्गोरिदम कहा जाता है। रिकर्सिव प्रोसीजर के कॉन्सेप्ट को सफलतापूर्वक लागू करने के लिए, नीचे दी गई शर्तें पूरी होनी चाहिए।

- (i) कंप्यूटर सिस्टम में इन-बिल्ट मैकेनिज्म होना चाहिए जो किसी प्रोसीजर को खुद से कॉल करने में मदद करता है, जैसे, स्टैक रजिस्टर के सेट पर इन-बिल्ट स्टैक ऑपरेशन हो सकते हैं।
- (ii) रिकर्सिव प्रोसीजर की परिभाषा में ऐसी शर्तें होनी चाहिए जिनके तहत, सीमित संख्या में कॉल के बाद, प्रोसीजर खत्म हो जाता है।
- (iii) क्रमिक कॉल में तर्क इस अर्थ में सरल होने चाहिए कि प्रत्येक आगामी तर्क हमें (ii) में वर्णित शर्तों की ओर ले जाता है।

रिकर्सन एक ज़रूरी कंस्ट्रक्ट है जिसका इस्तेमाल सॉर्टिंग एल्गोरिदम, सर्चिंग एल्गोरिदम, मैट्रिक्स मल्टिप्लिकेशन वगैरह को सॉल्व करने के लिए बड़े पैमाने पर किया जाएगा।

1.4 कॉमन रनिंग टाइम का सर्वे

किसी दी गई प्रॉब्लम के लिए, एक से ज़्यादा एल्गोरिदम डिज़ाइन किए जा सकते हैं। हालाँकि, एक एल्गोरिदम दूसरे से बेहतर हो सकता है। किसी प्रॉब्लम के लिए दो एल्गोरिदम की तुलना करने के लिए, आम तौर पर रनिंग टाइम का इस्तेमाल किया जाता है, जिसे आउटपुट बनाने में एल्गोरिदम द्वारा लिया गया समय माना जाता है। एक एल्गोरिदम बेहतर होता है अगर वह कम रनिंग टाइम लेता है। यहाँ "टाइम" ज़रूरी नहीं कि क्लॉक टाइम हो। हालाँकि, यह माप इस्तेमाल किए गए किसी भी हार्डवेयर के लिए इनवेरिएंट होना चाहिए। इसलिए, किसी एल्गोरिदम के रनिंग टाइम को किसी दिए गए इनपुट के लिए किए गए ऑपरेशन की संख्या के रूप में दिखाया जा सकता है। ऑपरेशन की संख्या जितनी ज़्यादा होगी, एल्गोरिदम का रनिंग टाइम उतना ही ज़्यादा होगा। इसलिए, अगर हम किसी एल्गोरिदम में दिए गए इनपुट के लिए ज़रूरी ऑपरेशन की संख्या पता कर सकते हैं तो हम रनिंग टाइम को माप सकते हैं। आउटपुट बनाने के लिए एल्गोरिदम के इस रनिंग टाइम को टाइम कॉम्प्लेक्सिटी भी कहा जाता है। यहाँ टाइम क्लॉक टाइम नहीं है।

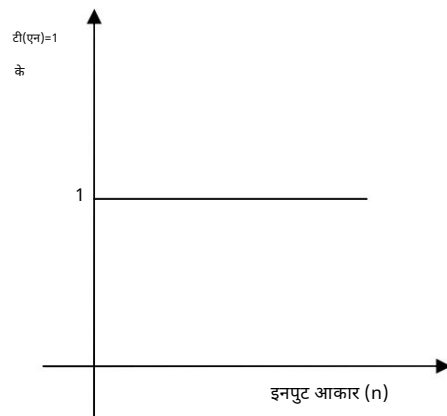
इसलिए, टाइम कॉम्प्लेक्सिटी के हिसाब से दो एल्गोरिदम की तुलना की जा सकती है। एक एल्गोरिदम दूसरों की तुलना में बेहतर है जिनका रनिंग टाइम (टाइम कॉम्प्लेक्सिटी) कम है।

किसी एल्गोरिदम का रनिंग टाइम एक फ़ंक्शन $T(n)$ के तौर पर दिखाया जाता है, जहाँ n इनपुट साइज़ है। मान लीजिए, किसी एल्गोरिदम का रनिंग टाइम $T(n) = cn$ है, जहाँ c एक कॉन्स्टेंट है। इस एल्गोरिदम का रनिंग टाइम इनपुट के साइज़ पर लीनियरली डिपेंड करता है। $T(n)$ की यूनिट अनस्पेसिफाइड है।

एल्गोरिदम के लिए रनिंग टाइम का सामान्य रूप नीचे दिया गया है:

1. स्थिर समय ($O(1)$): यदि चलने का समय इनपुट आकार (n) पर निर्भर नहीं करता है तो इसे स्थिर चलने का समय कहा जाता है। इसे इस प्रकार दर्शाया जा सकता है

एल्गोरिदम की मूल बातें और
इसके गुण



चित्र (a): $T(n) = O(1)$

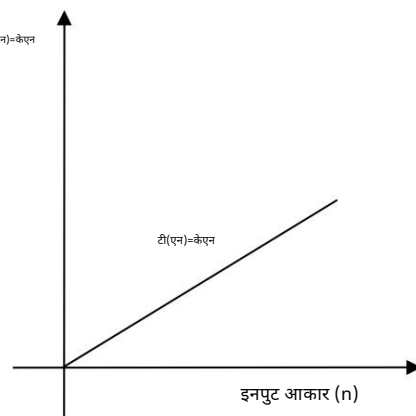
2. लीनियर टाइम $O(kn)$: अगर टाइम कॉम्प्लेक्सिटी ज़्यादा से ज़्यादा इनपुट के साइज़ का एक कॉन्स्टेंट फैक्टर गुना है, तो इसे लीनियर टाइम कॉम्प्लेक्सिटी कहा जाता है और इसे $T(n) \leq kn$ के तौर पर दिखाया जाता है, जहाँ k एक कॉन्स्टेंट है या $T(n) = O(n)$ । इस तरह की कॉम्प्लेक्सिटी वाला एल्गोरिदम आम तौर पर एक ही पास में एग्ज़िक्यूशन पूरा कर लेता है। उदाहरण के लिए, किसी दिए गए n की मिनिमम वैल्यू खोजना।

एक array में नंबर्स की प्रोसेसिंग सिर्फ़ एक पास में पूरी हो सकती है।

नीचे दिया गया प्रोग्राम का हिस्सा दिखाता है। इस तरह हम एक ऐरे के हर एलिमेंट को प्रोसेस करने में लगातार काम करते हैं।

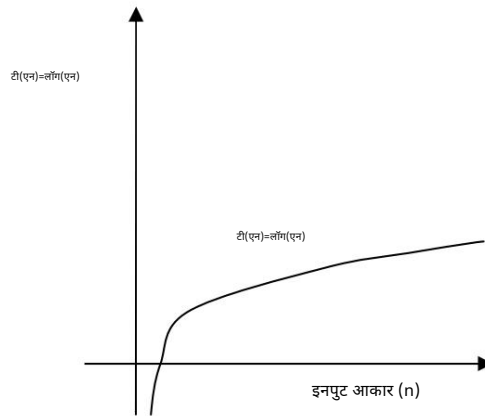
```

न्यूनतम = a[1]
i = 2 से n तक
यदि a[i] < न्यूनतम
    न्यूनतम = a[i]
अंत
अंत अगर
    
```



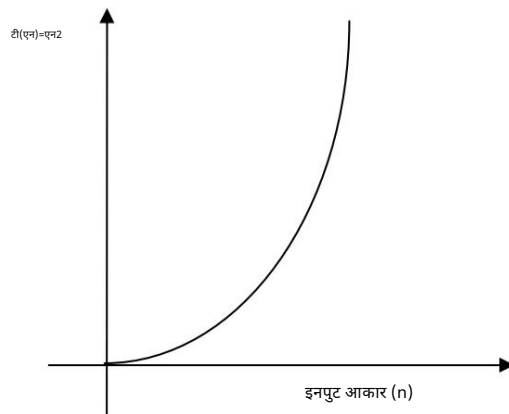
चित्र (b): $T(n) = O(n)$

3. लॉगरिदमिक टाइम ($\log(n)$): अगर किसी एल्गोरिदम की टाइम कॉम्प्लेक्सिटी इनपुट साइज़ के लॉगरिदम के प्रोपोर्शनल है, तो इसे लॉगरिदमिक टाइम कॉम्प्लेक्सिटी कहा जाता है और इसे $O(\log n)$ टाइम के तौर पर दिखाया जाता है। उदाहरण के लिए, बाइनरी सर्च एल्गोरिदम का रनिंग टाइम $O(\log n)$ है। $O(n \log n)$ कई एल्गोरिदम के लिए एक बहुत ही आम रनिंग टाइम है, जिन्हें डिवाइड एंड कॉन्कर टेक्नीक जैसे मर्ज सॉर्ट, क्विक सॉर्ट एल्गोरिदम, वगैरह से सॉल्व किया जाता है। इन सभी प्रॉब्लम में आम ऑपरेशन एरे को बराबर साइज़ के सब-एरे में स्प्लिट करना और फिर इसे रिकर्सिवली सॉल्व करना है।



चित्र (c): $T(n) = O(\log(n))$

क्वाड्रेटिक टाइम: ($T(n) = O(n^2)$) - यह तब होता है जब एल्गोरिदम में नेस्टेड लूप की एक जोड़ी होती है। बाहरी लूप $O(n)$ टाइम दोहराता है और हर दोहराव के लिए अंदरूनी लूप $O(n)$ टाइम लेता है, इसलिए हमें n के इन दो फैक्टर को गुणा करके $O(n^2)$ मिलता है। असल में यह छोटे इनपुट साइज़ या एलिमेंट्री सॉर्टिंग एल्गोरिदम की समस्या के लिए उपयोगी है। बबल सॉर्ट, इंसर्शन सॉर्ट, सिलेक्शन सॉर्ट और इंसर्शन सॉर्ट रनिंग टाइम कॉम्प्लेक्सिटी के लिए सबसे खराब केस टाइम कॉम्प्लेक्सिटी $O(n^2)$ है।



चित्र (d): $T(n) = O(n^2)$

चलने के समय की जटिलता के लिए हमें यह पता लगाना होगा कि कितने अलग-अलग

किसी सेट के n एलिमेंट्स के k साइज़ के सबसेट चुने जा सकते हैं। उसके लिए हम

एक समय में k तत्वों को लेकर n तत्वों का संयोजन लेना होगा।

उदाहरण के लिए, आइए एक ग्राफ में एक इंडिपेंडेंट सेट खोजने की समस्या पर विचार करें

जिसे नोड्स के एक सेट के रूप में परिभाषित किया जा सकता है जिसमें नोड्स की कोई जोड़ी नहीं है

उनके बीच एक किनारा है। आइए इंडिपेंडेंट सेट प्रॉब्लम को इस तरह बनाते हैं

निम्न तरीके से: एक स्थिरांक k और n नोड्स वाला एक ग्राफ G दिया गया है

(वर्तिस) साइज़ k का एक इंडिपेंडेंट सेट पता करें।

इस समस्या को हल करने के लिए ब्रूट फ़ोर्स तरीके से सर्च करना होगा

k नोड्स के सभी उपसमुच्चयों के लिए और प्रत्येक उपसमुच्चय के लिए यह जाँच करेगा कि क्या

साइज़ k के हर सबसेट s के लिए किन्हीं दो नोड्स को जोड़ने वाला एक किनारा होता है

नीचे एक स्वतंत्र सेट खोजने के लिए एक छद्म कोड है।

छद्म कोड

ग्राफ G में आकार k के प्रत्येक उपसमुच्चय s के लिए

जाँचें कि क्या s एक स्वतंत्र सेट है

यदि हाँ, तो प्रिंट करें “ s एक स्वतंत्र सेट है

वरना रुको

इस मामले में बाहरी लूप $O(nk)$ बार दोहराएगा और यह सभी k - को चुन लेगा

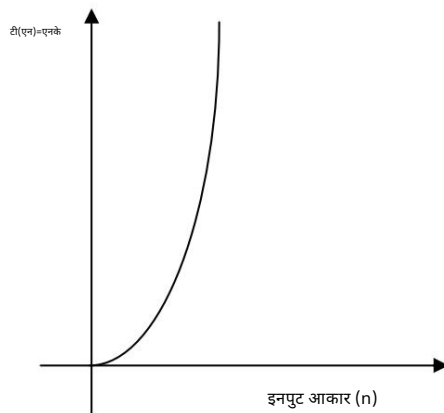
ग्राफ के n नोड के नोड सबसेट। हर एक के अंदर के लूप में

सबसेट यह नोड्स की प्रत्येक जोड़ी के लिए लूप करता है ताकि पता लगाया जा सके कि क्या कोई है

जोड़ी के बीच किनारा जिसके लिए $O(k^2)$ की आवश्यकता होगी

खोज के जोड़े यानी $O(k^2)$ सर्च करें। इसलिए अब कुल समय है

ओ(k^2 एन²) चूँकि k एक स्थिरांक है, इसे छोड़ा जा सकता है, अंत में यह $O(nk)$ है।



चित्र (f): $T(n) = O(nk)$

एल्गोरिदम की मूल बातें और
इसके गुण

6. घातांकीय समय: ($O(kn)$) बहुपद समय जटिलता से परे अन्य दो प्रकार की सीमाएँ हैं: घातांकीय समय $O(2N)$ और

फैक्टोरियल टाइम $O(n!)$: आइए हम इंडिपेंडेंट सेट प्रॉब्लम को रिफाइन करें जिसे हम

उन्हें n साइज़ का एक ग्राफ़ दिया गया है और वे a से स्वतंत्र ग्राफ़ ढूँढना चाहते हैं

किसी स्थिरांक k के बजाय अधिकतम मान जो n से कम है।

स्यूडो-कोड का मॉडिफाईड वर्शन नीचे दिया गया है।

स्यूडो-कोड : किसी ग्राफ का इंडिपेंडेंट सेट खोजने के लिए सूडो-कोड

इनपुट $G(V, E)$

{

n संख्या के नोड्स के प्रत्येक उपसमूह s के लिए

सत्यापित करें कि क्या s एक स्वतंत्र सेट है

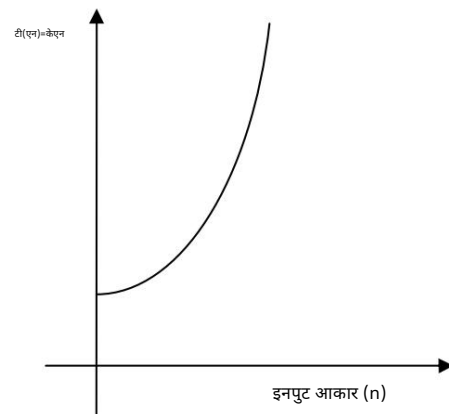
यदि s सभी जांचे गए उपसमुच्चयों में सबसे बड़ा है, तो

प्रिंट " s सबसे बड़ा स्वतंत्र सेट है

अगर अंत

अंत के लिए

}कोड खंड का अंत



चित्र (g): $T(n) = O(kn)$

इस मामले में n एलिमेंट्स के सबसेट्स की कुल संख्या 2^n होगी

बाहरी लूप times के बजाय 2 बार निष्पादित होगा

सबसेट्स के सभी पेयर्स यानी $(2n)$ का वेरिफिकेशन करें कि इन सबसेट्स में एज हैं या नहीं और फिर मैक्सिमम चुनना $O(n^2)$ होगा यानी सबसेट्स के पेयर्स की कुल संख्या। टोटल रनिंग टाइम होगा

$O(n^2 \cdot 2n) \cdot O(2n)$ रनिंग टाइम जटिलता तब उत्पन्न होती है जब एक खोज एल्गोरिथ्म n एलिमेंट्स के सभी सबसेट्स पर विचार करता है।

फैक्टोरियल टाइम ($O(n!)$): एक्सपोनेंशियल रनिंग टाइम की ग्रोथ की तुलना में, फैक्टोरियल टाइम ($n!$) की ग्रोथ ज़्यादा तेज़ होती है। इस तरह की कॉम्प्लेक्सिटी का रनिंग टाइम दो तरह के एल्गोरिदम में होता है:

(i) मिलान के लिए एल्गोरिदम, उदाहरण के लिए द्विदलीय मिलान एल्गोरिदम।

मान लीजिए n लड़के और n लड़कियाँ हैं। n लड़कों और n लड़कियों के बीच परफेक्ट मैचिंग ढूँढने के लिए, पहले लड़के की तुलना n लड़कियों से की जाएगी। दूसरे लड़के के पास तुलना के लिए लड़कियों में से $(n-1)$ ऑप्शन बचेंगे। तीसरे लड़के के लिए मैचिंग के लिए सिर्फ $(n-2)$ ऑप्शन होंगे, और इसी तरह आगे भी। लड़कियों को एरे करने के बाद, n लड़कों के लिए इन सभी ऑप्शन को गुणा करने पर हमें n मिलता है! यानी $n(n-1)(n-2) \dots (2)(1)$

(ii) $O(n!)$ तब भी होता है जब एल्गोरिदम को n एलिमेंट्स को एक खास ऑर्डर (यानी n नंबरों का क्रमचय) में अरेंज करने की ज़रूरत होती है। इसका एक क्लासिक उदाहरण ट्रैवलिंग सेल्समैन प्रॉब्लम है। शहरों के सभी जोड़ों के बीच दूरी वाले कुछ शहर दिए गए हैं, जिनमें ये शर्तें हैं: (i) सेल्समैन किसी भी शहर से दूर शुरू कर सकता है, लेकिन दूर को शुरूआती शहर से ही खत्म करना होगा (ii) दूर शुरू होने वाले शहर को छोड़कर सभी शहरों में सिर्फ एक बार जाना होगा। प्रॉब्लम यह है कि सभी n शहरों को कवर करने वाला सबसे छोटा दूर ढूँढना है। सॉल्यूशन ढूँढने के लिए ब्रूट फ़ोर्स अप्रोच का इस्तेमाल करते हुए, एक सेल्समैन को $n!$ सर्च करने होंगे जिसमें $O(n!)$ लगेगा। ध्यान दें कि एक सेल्समैन दूर शुरू करने के लिए n शहरों में से कोई भी शहर चुन सकता है। इसके बाद उसके पास दूर पर दूसरे शहर को चुनने के लिए $(n-1)$ शहर होंगे। अगले स्टेज पर तीसरे शहर को चुनने के लिए $(n-2)$ शहर होंगे और इसी तरह आगे भी। इन सभी चॉइस को गुणा करने पर हमें $n!$ यानी $n(n-1)(n-2) \dots (2)(1)$ मिलता है।

1.5 एल्गोरिदम का एनालिसिस और कॉम्प्लेक्सिटी

"एनालिसिस ऑफ़ एल्गोरिदम" शब्द डोनाल्ड नुथ ने शुरू किया था। यह अब एक ज़रूरी कंप्यूटर साइंस का फ़िल्ड बन गया है जिसका पूरा मकसद टाइम कॉम्प्लेक्सिटी और स्टोरेज की ज़रूरत के हिसाब से एल्गोरिदम की कॉम्प्लेक्सिटी को समझना है।

सिस्टम परफॉर्मेंस सीधे तौर पर टाइम कॉम्प्लेक्सिटी और मेमोरी दोनों के मामले में एल्गोरिदम की एफिशिएंसी पर निर्भर करती है। टाइम सेंसिटिव एप्लिकेशन के लिए डिज़ाइन किया गया एल्गोरिदम अगर चलने में बहुत ज़्यादा समय लेता है, तो उसके नतीजे बेकार हो सकते हैं।

उपयोग।

मान लीजिए M एक एल्गोरिदम है जिसका इनपुट डेटा साइज़ n है। एल्गोरिदम M द्वारा इस्तेमाल किया गया समय और जगह, M की एफिशिएंसी के दो मुख्य माप हैं। समय को मुख्य ऑपरेशन की संख्या गिनकर मापा जाता है, उदाहरण के लिए, सॉर्टिंग और सर्चिंग एल्गोरिदम के मामले में, तुलना की संख्या होती है।

होज एल्गोरिथ की जटिलता x और सरणी तत्वों $A[K]$ के बीच तुलनाओं की संख्या C द्वारा दी गई x और सरणी तत्व $A[K]$ के बीच तुलना।

A. इस मामले में $x = A[LOC]$. $C(n) = 1$

$$C(n) = n.$$

होने की प्रत्येक संभावना को दर्शाती है। हम संभावना की संख्या, array में किसी भी पोजीशन पर होने की बराबर संभावना रखती है। यहां तुलना की $1, 2, 3, \dots, n$, और प्रत्येक संख्या $p=1/n$ की संभावना के साथ आती है तो

$$\begin{aligned} C(n) &= 1 \cdot \frac{1}{n} + 2 \cdot \frac{1}{n} + \dots + n \cdot \frac{1}{n} \\ &= (1 + 2 + \dots + n) \cdot \frac{1}{n} \\ &= \frac{n(n+1)}{2} \cdot \frac{1}{n} = \frac{n+1}{2} \end{aligned}$$

३३८

(अधिकतम चरण)] को व्यक्त करने के लिए एक विषय प्रस्ताव है। समस्या को हल करने के लिए आवश्यक ऊपरी सीमा

(कम से कम) कदम को व्यक्त करने के लिए स्विचिंग के लिए उपयोग निचली सीमा अर्थात किसी समस्या को हल करने के लिए आवश्यक न्यूनतम

- Θ ('थीटा') संकेतन. [उपरी और निचरी दोनों सीमाओं को बंधन के लिए इसी मोल होता है, जिसका डाइट बाउंड भी कहते हैं।]

"काफी बड़े इनपुट साइज़" के लिए, किसी खास इनपुट साइज़ (i.e. $n \rightarrow \infty$)

नोटेशन का इस्तेमाल अपर बाउंड (सबसे खराब केस) को दिखाने के लिए किया जाता है; Ω -

नौ एशमोटेकान हबो इस्तेमाल साधन बाउंड (संड स्पेस सेडमकेस) केसे दिखे दिखे के निम्न किश किशता हैत है और ०- नोटेन का इस्तेमाल अतिशय शो अर दोरी और इंड (ग्राफीड (यानी टाइट बाउंड) दोनों को एक्सप्रेस करें।

एल्गोरिदम की मूल बातें और
इसके गुण

हम आम तौर पर अपने फ़ंक्शन की ग्रोथ के लिए एसिम्प्टोटिक लोअर बाउंड और अपर बाउंड या तो या दोनों को ढूँढना चाहते हैं।

1.6 समस्याओं के प्रकार

कंप्यूटिंग में प्रॉब्लम के टाइप अनगिनत हैं, और उन्हें कुछ एरिया में बांटा गया है ताकि रिसर्चर के लिए एल्गोरिदम फील्ड को देखते हुए प्रॉब्लम के टाइप को समझना आसान हो जाए।

कुछ आम तौर पर जाने-पहचाने प्रॉब्लम टाइप नीचे दिए गए हैं:

- सॉर्टिंग •
- सर्चिंग • ग्राफ़
- प्रॉब्लम • कॉम्बिनेटरियल
- प्रॉब्लम • जियोमेट्रिक प्रॉब्लम • न्यूमेरिकल
- प्रॉब्लम

ऊपर बताई गई कैटेगरी के लिए, एल्गोरिदम को एनालाइज़ करने के लिए कुछ स्टैंडर्ड इनपुट सेट को बेंचमार्किंग सेट के तौर पर डिफाइन किया गया है।

छंटार्ड

सॉर्टिंग वह प्रोसेस है जिसमें दिए गए आइटम के सेट को एक खास ऑर्डर में अरेंज किया जाता है, यह मानते हुए कि आइटम का नेचर ऐसा ऑर्डर करने की इजाज़त देता है। उदाहरण के लिए, नंबरों के सेट को बढ़ते या घटते ऑर्डर में सॉर्ट करना और कैरेक्टर स्ट्रिंग, जैसे नाम, को अल्फाबेटिकल ऑर्डर में सॉर्ट करना।

रिसर्चर्स ने अलग-अलग तरह के आइटम्स को टारगेट करते हुए कई अलग-अलग सॉर्टिंग एल्गोरिदम पब्लिश किए हैं। एक सॉर्टिंग एल्गोरिदम ज़रूरी नहीं कि सभी तरह के आइटम्स की लिस्ट के लिए सबसे अच्छे से काम करे। उनमें से कुछ रिसोर्स इस्तेमाल के मामले में अच्छे हैं, जबकि कुछ कंप्यूटिंग के मामले में तेज़ हैं। सॉर्टिंग एल्गोरिदम की एफिशिएंसी इनपुट के टाइप पर भी निर्भर करती है, कुछ रैंडमली ऑर्डर किए गए इनपुट पर अच्छा काम करते हैं, जबकि दूसरे पहले से लगभग सॉर्ट की गई लिस्ट्स पर बेहतर परफॉर्म करते हैं। कुछ सॉर्टिंग एल्गोरिदम मेमोरी में रहने वाली लिस्ट्स के लिए अच्छा परफॉर्म करते हैं, जबकि दूसरे सेकेंडरी डिस्क पर स्टोर बड़ी फाइलों को सॉर्ट करने के लिए सबसे अच्छे से काम करते हैं।

अभी तक आम तौर पर, सबसे अच्छा सॉर्टिंग एल्गोरिदम n आइटम की आइटम लिस्ट को सॉर्ट करने के लिए $n \log n$ तुलना लेता है।

किसी भी सॉर्टिंग एल्गोरिदम के लिए ये दो खासियतें ज़रूरी हैं:

1. स्थिरता 2.
- यथास्थान.

एक सॉर्टिंग एल्गोरिदम को स्टेबल कहा जाता है अगर यह इनपुट लिस्ट के किन्हीं दो बराबर आइटम की रिलेटिव पोज़िशन को नहीं बदलता है। मान लीजिए, एक इनपुट लिस्ट में, दो बराबर आइटम हैं।

आइटम की पोजीशन i और j है, जहाँ $i < j$ है, तो सॉर्टेड लिस्ट में इन आइटम की आखिरी पोजीशन भी एक के बाद एक k और $k+1$ होनी चाहिए, ताकि $k < i$ हो। यानी इन बराबर आइटम के बीच कोई अदला-बदली नहीं होनी चाहिए और उनकी पोजीशन एक-दूसरे से बदलनी नहीं चाहिए।

स्वैपिंग प्रोसेस के दौरान एलिमेंट्स को स्टोर करने के लिए सॉर्टिंग एल्गोरिदम को एक्स्ट्रा मेमोरी स्पेस की ज़रूरत होती है। लिस्ट में आइटम्स के छोटे सेट के लिए, यह कंस्ट्रेंट ऑब्ज़र्वेबल नहीं होता है, लेकिन बड़े एलिमेंट्स की इनपुट लिस्ट के लिए ज़रूरी स्टोरेज स्पेस काफ़ी बड़ा होता है। एक एल्गोरिदम को इन-प्लेस कहा जाता है अगर ज़रूरी एक्स्ट्रा मेमोरी मार्केबल न हो।

खोज

सर्चिंग का मतलब है किसी दिए गए आइटम के सेट में एक एलिमेंट ढूँढना, जिसे सर्च की कहते हैं (इसकी वैल्यू रिडंडेंट हो सकती है)। सर्चिंग किसी भी डेटासेट/डेटाबेस पर सबसे ज़रूरी और अक्सर किया जाने वाला ऑपरेशन है।

एल्गोरिदम एनालिसिस के फील्ड में सर्चिंग रिसर्च के सबसे पसंदीदा एरिया में से एक है। कोई भी एक सर्चिंग सभी सिचुएशन में सबसे अच्छा काम नहीं करती है।

कुछ एल्गोरिदम तेज़ होते हैं लेकिन ज़्यादा मेमोरी इस्तेमाल करते हैं; कुछ बहुत तेज़ होते हैं लेकिन सिर्फ़ खास इनपुट सेट के साथ; वगैरह।

सर्चिंग प्रॉब्लम के लिए एल्गोरिदम डिज़ाइन करते समय, यह अंदरूनी डेटा के नेचर से बहुत ज़्यादा प्रभावित होता है। डेटा, जो स्टैटिक होता है, उसे डायनामिक नेचर के डेटा सेट से किसी आइटम को जोड़ने या हटाने के साथ अलग तरह से एड्रेस करना होता है।

स्ट्रिंग प्रसंस्करण

सोशल मीडिया और ब्लॉग पर अलग-अलग एप्लिकेशन की वजह से टेक्स्ट डेटा में तेज़ी से बढ़ोतरी हुई है, स्ट्रिंग-हैंडलिंग एल्गोरिदम रिसर्च का एक मौजूदा एरिया बन गया है। टेक्स्ट प्रोसेसिंग के बजाय स्ट्रिंग्स के बढ़ने का एक और कारण उपलब्ध डेटा का प्रकार है। आजकल बिज़नेस के तरीके ऑफ़लाइन से ऑनलाइन में पूरी तरह बदल गए हैं। ग्रांट थॉर्नटन के अनुसार, भारत में ई-कॉमर्स 2025 तक US\$ 188 बिलियन का होने की उम्मीद है। ज़्यादातर टेक्स्ट डेटा का इस्तेमाल कमर्शियल ऑर्गनाइज़ेशन, खासकर ई-कॉमर्स सेक्टर के लिए डायरेक्ट या इनडायरेक्ट मॉनेटरी फ़ायदे से जुड़े लोगों की दिलचस्पी का अनुमान लगाने के लिए किया जाता है। सबसे ज़्यादा इस्तेमाल होने वाले सर्च इंजन (गूगल) में से एक भी स्ट्रिंग प्रोसेसिंग पर आधारित है।

स्ट्रिंग मैचिंग स्ट्रिंग प्रोसेसिंग समस्याओं में से एक है।

ग्राफ़ समस्याएँ

रिसर्चर्स के लिए कम्प्यूटेशनल प्रॉब्लम को ग्राफ़ प्रॉब्लम से मैप करना हमेशा फ़ायदेमंद होता है। कई कम्प्यूटेशनल प्रॉब्लम को ग्राफ़ का इस्तेमाल करके हल किया जा सकता है।

ज़्यादातर कंप्यूटर नेटवर्क की समस्याओं को ग्राफ़ एल्गोरिदम का इस्तेमाल करके अच्छे से हल किया जा सकता है। जैसे: ग्राफ़ के सभी नोड्स पर जाना (नेटवर्क में ब्रॉडकास्टिंग), नेटवर्क में रूटिंग (मिनिमम कॉस्ट पाथ ढूँढना, यानी सबसे छोटा रास्ता)

एल्गोरिदम की मूल बातें और
इसके गुण

रास्ता, कम से कम देरी वाला रास्ता वगैरह को ग्राफ एल्गोरिदम से अच्छे से हल किया जा सकता है।

वहीं, कुछ ग्राफ प्रॉब्लम कम्प्यूटेशन के हिसाब से आसान नहीं होतीं, जैसे ट्रैवलिंग सेल्समैन और ग्राफ-कलरिंग प्रॉब्लम। ट्रैवलिंग सेल्समैन प्रॉब्लम (TSP) का इस्तेमाल n शहरों को कवर करने के लिए किया जाता है, जिसमें सबसे छोटा रास्ता लिया जाता है और किसी भी शहर में एक से ज्यादा बार नहीं जाया जाता। ग्राफ-कलरिंग प्रॉब्लम में ग्राफ के सभी वर्टिसेस को कम से कम रंगों से कलर करने की कोशिश की जाती है, ताकि कोई भी दो आस-पास के वर्टिसेस एक जैसे रंग के न हों। TSP को सॉल्व करते समय शहरों को ग्राफ के वर्टिसेस माना जा सकता है। इवेंट शेड्यूलिंग उन प्रॉब्लम में से एक हो सकती है जिसे ग्राफ कलरिंग एल्गोरिदम का इस्तेमाल करके सॉल्व किया जा सकता है। इवेंट्स को वर्टिसेस से दिखाने पर विचार करते हुए, दो इवेंट्स के बीच एक एज तभी होता है जब संबंधित इवेंट्स को एक ही समय पर शेड्यूल नहीं किया जा सकता।

संयोजनात्मक समस्याएँ

इस तरह की प्रॉब्लम में सॉल्यूशन का कॉम्बिनेशन होता है, यानी एक से ज्यादा सॉल्यूशन हो सकते हैं। कॉम्बिनेटरियल प्रॉब्लम का मकसद दी गई कंडीशन को पूरा करने वाले परम्यूटेशन, कॉम्बिनेशन या सबसेट ढूँढना है। ट्रैवलिंग सेल्समैन प्रॉब्लम, इंडिपेंडेंट सेट और ग्राफ-कलरिंग प्रॉब्लम को कॉम्बिनेटरियल प्रॉब्लम के उदाहरण के तौर पर कैटेगरी में रखा जा सकता है। थ्योरेटिकल और प्रैक्टिकल, दोनों नज़रिए से, कॉम्बिनेटरियल प्रॉब्लम को कंप्यूटिंग में सबसे मुश्किल प्रॉब्लम में से एक माना जाता है। कॉम्बिनेटरियल टाइप के सॉल्यूशन की वजह से, बड़े साइज़ के इनपुट सेट वाली प्रॉब्लम को हँडल करना बहुत मुश्किल हो जाता है। प्रॉब्लम के साइज़ के साथ कॉम्बिनेटरियल ऑब्जेक्ट (आउटपुट सॉल्यूशन) की संख्या तेज़ी से बढ़ती है।

ज्यामितीय समस्याएं

जियोमेट्रिक एल्गोरिदम के कुछ इस्तेमाल कंप्यूटर ग्राफिक्स, रोबोटिक्स और टोमोग्राफी हैं। ये एल्गोरिदम पॉइंट्स, लाइन्स और पॉलीगॉन जैसी जियोमेट्रिक चीज़ों पर आधारित होते हैं। ज्योमेट्री के तरीके अलग-अलग जियोमेट्रिक समस्याओं को हल करने के लिए बनाए गए हैं, जैसे रूलर और कंपास का इस्तेमाल करके ज्योमेट्रिक चीज़ों, ट्रायंगल, सर्कल वगैरह के शेष बनाना।

कम्प्यूटेशनल ज्योमेट्री की जानी-मानी क्लासिक प्रॉब्लम ये हैं:

1. निकटतम-युग्म समस्या
2. उत्तल-पतवार समस्या

सबसे करीबी जोड़ी की समस्या प्लेन में दिए गए पॉइंट्स के सेट में से सबसे करीबी जोड़ी को ढूँढना है।

कॉन्वेक्स-हल प्रॉब्लम में, सबसे छोटा कॉन्वेक्स पॉलीगॉन बनाना होता है ताकि उसमें दिए गए सेट के सभी पॉइंट शामिल हों।

संख्यात्मक समस्याएं

न्यूमेरिकल कंप्यूटिंग नेचर की प्रॉब्लम हैं सिमुलटेनियस लीनियर इक्वेशन (लीनियर अलजेब्रा), डिफरेंशियल इक्वेशन, डेफिनिट इंटीग्रेशन और स्टैटिस्टिक्स।
ज्यादातर न्यूमेरिकल प्रॉब्लम लगभग हल की जा सकती थीं।

न्यूमेरिकल एल्गोरिदम की सबसे बड़ी कमी यह है कि कई इटरेशन में गलतियाँ जमा हो जाती हैं, क्योंकि हर इटरेशन पर अनुमानित नतीजे को राउंड ऑफ़ कर दिया जाता है।

1.7 समस्या समाधान तकनीकें

फूट डालो और राज करो दृष्टिकोण

यह एक पॉपुलर तरीका है जिसमें किसी प्रॉब्लम को छोटी सब-प्रॉब्लम में बांटा जाता है। इन सब-प्रॉब्लम को तब तक और छोटी सब-प्रॉब्लम में बांटा जाता है जब तक कि उन्हें और बांटा न जा सके। यह एक टॉप-डाउन तरीका है जिसमें एल्गोरिदम लॉजिकली शुरुआती इंस्टेंस से लेकर बीच के सब-इंस्टेंस के ज़रिए सबसे छोटे सब-इंस्टेंस तक आगे बढ़ता है।

डिवाइड एंड कॉन्कर टेक्निक को फॉलो करने वाले एल्गोरिदम में ये स्टेप्स होते हैं:

- स्टेप 1. प्रॉब्लम (टॉप लेवल) को सब-प्रॉब्लम (लोअर लेवल) के सेट में बाँट दें।
स्तर)।
- स्टेप 2. हर सब-प्रॉब्लम को रिकर्सिव अप्रोच से अलग-अलग सॉल्व करें।
- स्टेप 3. सब-प्रॉब्लम के सॉल्यूशन को प्रॉब्लम के पूरे सॉल्यूशन में मिला दें।

नीचे उन समस्याओं के उदाहरण दिए गए हैं जिन्हें डिवाइड एंड कॉन्कर अप्रोच का इस्तेमाल करके अच्छे से हल किया जा सकता है।

- बाइनरी सर्च। • क्विक सॉर्ट। •
- मर्ज सॉर्ट। • स्ट्रेसन मैट्रिक्स
- मल्टिप्लिकेशन। • पॉइंट्स
- का सबसे करीबी जोड़ा।

लालची तकनीक

ग्रीडी अप्रोच का इस्तेमाल करके, ऑप्टिमाइज़ेशन प्रॉब्लम को अच्छे से हल किया जाता है। ऑप्टिमाइज़ेशन प्रॉब्लम में, दिए गए इनपुट वैल्यू को या तो मैक्सिमाइज़ या मिनिमाइज़ (जिसे ऑब्जेक्टिव कहते हैं) करना होता है, जो कुछ कंडीशन या शर्तों के अधीन होता है।

- Greedy एल्गोरिदम हमेशा किसी खास समय पर किसी दिए गए मकसद को ऑप्टिमाइज़ करने के लिए कई ऑप्शन में से सबसे अच्छा ऑप्शन (greedy approach) चुनता है।

एल्गोरिदम की मूल बातें और
इसके गुण

- ग्रीडी मेथड हर स्टेप पर लोकल ऑप्टिमम चुनता है और इस फैसले का नतीजा ओवरऑल नॉन-ऑप्टिमम या ऑप्टिमम सॉल्यूशन हो सकता है। • ग्रीडी अप्रोच हमेशा ऑप्टिमल सॉल्यूशन नहीं देता, बल्कि ऑप्टिमल सॉल्यूशन के बहुत पास का सॉल्यूशन देता है।

इसलिए, ऑप्टिमाइज़ेशन प्रॉब्लम के लिए Greedy एल्गोरिदम को डिज़ाइन करना अक्सर बहुत आसान होता है। Greedy अप्रोच के कुछ उदाहरण नीचे दिए गए हैं।

- क्रुस्कल का मिनिमम स्पैनिंग ट्री • प्राइम का मिनिमल स्पैनिंग ट्री • डिज्कस्ट्रा का सबसे छोटा रास्ता • नैप्सैक प्रॉब्लम

डायनेमिक प्रोग्रामिंग

डायनामिक प्रोग्रामिंग अप्रोच एक बॉटम-अप अप्रोच है जिसमें सभी सब-प्रॉब्लम का सॉल्यूशन ढूँढना, इन पार्शियल रिज़ल्ट को सेव करना, और फिर बड़ी सब-प्रॉब्लम को सॉल्व करने के लिए उनका दोबारा इस्तेमाल करना शामिल है, जब तक कि ओरिजिनल प्रॉब्लम का सॉल्यूशन न मिल जाए। सब-प्रॉब्लम के रिज़ल्ट का दोबारा इस्तेमाल करना (रिज़ल्ट की एक टेबल बनाकर) डायनामिक प्रोग्रामिंग का सबसे बड़ा फ़ायदा है क्योंकि यह एक ही प्रॉब्लम के री-कम्प्यूटेशन (दो या ज़्यादा बार रिज़ल्ट कम्प्यूट करना) से बचाता है। इस तरह डायनामिक प्रोग्रामिंग अप्रोच में आसान या सीधे-सादे तरीकों की तुलना में बहुत कम समय लगता है।

डायनामिक प्रोग्रामिंग का काम करने का तरीका डिवाइड एंड कॉन्कर अप्रोच जैसा ही है। दोनों ही एक प्रॉब्लम को कई सब-प्रॉब्लम में तोड़कर सॉल्व करते हैं जिन्हें रिकर्सिवली सॉल्व किया जा सकता है। डिवाइड एंड कॉन्कर मेथड की कमी यह है कि इसमें एक ही आउटपुट/रिज़ल्ट वाले रिकर्सिव फंक्शन को बार-बार कॉल करना पड़ता है, जिसे डायनामिक प्रोग्रामिंग में रिज़ल्ट स्टोर करने के लिए एक टेबल बनाकर दूर किया जाता है। यह डायनामिकली तय होता है कि फंक्शन को कॉल करना है या टेबल से वैल्यू निकालनी है, इसीलिए इसमें डायनामिक शब्द का इस्तेमाल किया जाता है। डायनामिक प्रोग्रामिंग अप्रोच डिवाइड एंड कॉन्कर मेथड से ज़्यादा तेज़ है क्योंकि इसमें एक ही रिज़ल्ट वाले फंक्शन को कॉल करने की ज़रूरत नहीं होती है। 0-1 नैप्सैक और सबसेट-सम प्रॉब्लम डायनामिक प्रोग्रामिंग के उदाहरण हैं।

शाखा और सीमा

ब्रांच और बाउंड एल्गोरिदम डिस्क्रीट और कॉम्बिनेटोरियल ऑप्टिमाइज़ेशन प्रॉब्लम को अच्छे से सॉल्व करता है। ब्रांच-एंड-बाउंड एल्गोरिदम में, रूट पर पूरा सॉल्यूशन सेट के साथ एक रूटेड ट्री बनता है। एल्गोरिदम इस ट्री की ब्रांच को एक्सप्लोर करता है, जो सॉल्यूशन सेट के सबसेट को दिखाता है। रूट नोड का कैंडिडेट सॉल्यूशन तभी ब्रांच माना जाता है जब वह पहले से एक्सप्लोर किए गए सॉल्यूशन से बेहतर हो, और अगर वह एल्गोरिदम द्वारा अब तक पाए गए सबसे अच्छे सॉल्यूशन से बेहतर सॉल्यूशन नहीं दे पाता है तो उसे हटा दिया जाता है। ब्रांच और बाउंड एल्गोरिदम मेथड हैं।

ग्लोबल ऑप्टिमाइज़ेशन प्रॉब्लम को सॉल्व करने के लिए। हालाँकि, यह बहुत धीमा है। असल में, यह अक्सर सबसे खराब स्थिति में एक्सपोनेंशियल टाइम कॉम्प्लेक्सिटी की ओर ले जाता है। दूसरी ओर, अगर इसे ध्यान से लागू किया जाए, तो यह ऐसे एल्गोरिदम बना सकता है जो औसतन काफी तेज़ी से चलते हैं। B&B का आम आइडिया सबसे अच्छे सॉल्यूशन के लिए BFS जैसी खोज है, लेकिन सभी नोड्स एक्सपैंड नहीं होते (यानी, उनके बच्चे जेनरेट नहीं होते)।

यादृच्छिक एल्गोरिदम

रैंडमाइज्ड एल्गोरिदम में, सॉल्यूशन के किसी भी स्टेज पर एक रैंडम नंबर चुना जाता है और सॉल्यूशन के कैलकुलेशन के लिए इस्तेमाल किया जाता है, इसीलिए इसे रैंडमाइज्ड एल्गोरिदम कहा जाता है। दूसरे शब्दों में यह कहा जा सकता है कि जो एल्गोरिदम तेज़ सॉल्यूशन के लिए रैंडम चॉइस करते हैं, उन्हें रैंडमाइज्ड एल्गोरिदम कहा जाता है। उदाहरण के लिए, क्विक सॉर्ट एल्गोरिदम में, एक रैंडम नंबर जेनरेट किया जा सकता है और उसे पिवट माना जा सकता है। दूसरे उदाहरण में, एक रैंडम नंबर को एक बड़े नंबर को फैक्टर करने के लिए पॉसिबल डिवाइज़र के तौर पर चुना जा सकता है।

बैकट्रैकिंग एल्गोरिदम

बैकट्रैकिंग एल्गोरिदम नए सॉल्यूशन खोजते समय चेकपॉइंट बनाने जैसा है। यह डेप्थ-फर्स्ट सर्च की तरह काम करता है। यह सभी पॉसिबल सॉल्यूशन खोजता है। सॉल्यूशन खोजते समय, अगर कोई सॉल्यूशन काम नहीं करता है, तो यह पिछली जगह पर वापस जाता है और फिर सॉल्यूशन पाने के लिए दूसरे ऑप्शन ढूँढता है। अगर कोई और चॉइस पॉइंट नहीं हैं तो सर्च फेल हो जाती है।

1.8 नियतात्मक और स्टोकेस्टिक एल्गोरिथ्म

एल्गोरिदम को नेचर में या तो डिटरमिनिस्टिक या स्टोकेस्टिक कैटेगरी में बांटा जा सकता है। एक एल्गोरिदम डिटरमिनिस्टिक होता है अगर अगले आउटपुट को इनपुट और प्रोग्राम की स्थिति से प्रेडिक्ट/डिटर किया जा सके, जबकि स्टोकेस्टिक एल्गोरिदम नेचर में रैंडम होते हैं। अनप्रेडिक्टेबल रिजल्ट वाली प्रॉब्लम को डिटरमिनिस्टिक अप्रोच का इस्तेमाल करके सॉल्व नहीं किया जा सकता है। उदाहरण के लिए, ब्लैकजैक गेम के कार्ड शफ़लिंग प्रोग्राम का अगला आउटपुट प्लेयर्स द्वारा प्रेडिक्टेबल नहीं होना चाहिए, भले ही प्रोग्राम का सोर्स कोड दिखाई दे रहा हो। स््यूडो-रैंडम नंबर जनरेटर मेथड कॉम्प्रोमाइज़ किया जा सकता है और अक्सर असली रैंडमनेस पक्का करने के लिए काफी नहीं होता है। स््यूडो-रैंडम नंबर जनरेटर मेथड का इस्तेमाल करके जेनरेट किया गया रैंडम नंबर सटीक रूप से प्रेडिक्ट किया जा सकता है। इस प्रॉब्लम से बचने के लिए, जनरेटर को इनिशियलाइज़ करने के लिए अनप्रेडिक्टेबल रैंडम सीड के साथ क्रिप्टोग्राफिक रूप से सिक्कोर स््यूडो-रैंडम नंबर जनरेटर का इस्तेमाल बेहतर तरीका हो सकता है।

असली रैंडमनेस पाने के लिए हार्डवेयर रैंडम नंबर जनरेटर सबसे अच्छा तरीका है।

1.10 सारांश

एल्गोरिदम की मूल बातें और
इसके गुण

कम्प्यूटेशन में, एक एल्गोरिदम प्रोग्रामिंग भाषा से स्वतंत्र होता है।

एल्गोरिदम को कम्प्यूटेशनल प्रॉब्लम के सॉल्यूशन को समझने और एनालाइज़ करने के लिए डिज़ाइन किया गया है। एल्गोरिदम में, स्टेटमेंट का इस्तेमाल सीक्वेंसिंग नाम का एक्शन करने, या फैसला (सिलेक्शन) लेने या कुछ एक्शन (इटरेशन) दोहराने के लिए किया जा सकता है।

किसी एल्गोरिदम का रनिंग टाइम, एल्गोरिदम को आंकने के लिए सबसे ज़्यादा इस्तेमाल होने वाले पैरामीटर में से एक है। रनिंग टाइम को एग्जीक्यूट किए गए इंस्ट्रक्शन की संख्या के तौर पर कैलकुलेट किया जाता है। स्पेस कॉम्प्लेक्सिटी एग्जीक्यूशन के दौरान इस्तेमाल होने वाले मेमोरी स्टोरेज का मेज़रमेंट है। किसी कम्प्यूटेशनल प्रॉब्लम के लिए, सॉल्व करने के लिए कई एल्गोरिदम हो सकते हैं, जिससे ज़रूरत के हिसाब से सबसे अच्छा सॉल्यूशन पाने के लिए इन एल्गोरिदम को एनालाइज़ करना पड़ता है।

एक एल्गोरिदम को टाइम कॉम्प्लेक्सिटी और स्पेस कॉम्प्लेक्सिटी के हिसाब से एनालाइज़ किया जा सकता है। किसी प्रॉब्लम के एल्गोरिदम को डेवल्यूएट करने के लिए, टाइम और स्पेस कॉम्प्लेक्सिटी पर विचार किया जाता है। ऐसा एल्गोरिदम, जो मनचाहा आउटपुट देने में कम समय ले, अच्छा होता है। इन दोनों पैरामीटर के बीच एक ट्रेडऑफ़ होना चाहिए, कम स्पेस कॉम्प्लेक्सिटी वाला एल्गोरिदम शायद अच्छा न हो अगर वह लंबे समय तक चलता है।

कंप्यूटिंग में, प्रॉब्लम के नेचर के आधार पर, इसे आम तौर पर जानी-मानी कैटेगरी में से कोई भी दी जा सकती है, जैसे सॉर्टिंग, सर्चिंग, स्ट्रिंग प्रोसेसिंग, ग्राफ प्रॉब्लम, कॉम्बिनेटोरियल प्रॉब्लम, ज्योमेट्रिक प्रॉब्लम, न्यूमेरिकल प्रॉब्लम।

एक जैसी समस्याओं को एक जैसे तरीके से हल किया जा सकता है। आमतौर पर इस्तेमाल होने वाली कुछ समस्याएँ हल करने की तकनीकें हैं ब्रूट फ़ोर्स और एग्ज़ॉस्टिव सर्च अप्रोच, डिवाइड एंड कॉन्कर अप्रोच, ग्रीडी टेक्निक, डायनामिक प्रोग्रामिंग, ब्रांच एंड बाउंड, रैंडमाइज़्ड एल्गोरिदम, और बैकट्रैकिंग एल्गोरिदम।

इस्तेमाल किए गए इनपुट के टाइप के आधार पर एल्गोरिदम की एक और जानी-मानी कैटेगरी है डिटरमिनिस्टिक और स्टोकेस्टिक एल्गोरिदम। अगर इनपुट को देखकर किसी एल्गोरिदम का आउटपुट पता लगाया जा सकता है, तो ऐसे एल्गोरिदम को डिटरमिनिस्टिक कहा जाता है। जबकि स्टोकेस्टिक एल्गोरिदम रैंडम होते हैं, इसका मतलब है कि इनपुट से आउटपुट पता नहीं लगाया जा सकता।

1.11 अपनी प्रोग्रेस चेक करने का सॉल्यूशन

Q1. एल्गोरिदम क्या है? एल्गोरिदम की ज़रूरी खासियतें क्या हैं? जवाब: एल्गोरिदम किसी प्रॉब्लम या प्रॉब्लम के सेट को सॉल्व करने के लिए स्टेप्स का एक सेट है। साथ ही, एल्गोरिदम लॉजिकल और मैथमेटिकल या कम्प्यूटेशनल प्रॉब्लम को सॉल्व करने का एक स्टेप बाय स्टेप तरीका है। रेसिपी एल्गोरिदम का एक अच्छा उदाहरण है। कोई डिश बनाने के लिए रेसिपी बताती है कि स्टेप बाय स्टेप क्या करना है।

एल्गोरिदम की ज़रूरी खासियतें हैं: इनपुट, आउटपुट, निश्चितता, असर और सीमितता

Q2. एल्गोरिदम के बिल्डिंग ब्लॉक्स क्या हैं?

चयन, चयन, पुनरावृत्ति, प्रक्रिया और पुनरावृत्ति

Q3. किसी एल्गोरिदम को कैसे परखा जाए कि वह एफिशिएंट है या नहीं?

समाधान: एक एल्गोरिदम सही और कुशल दोनों होना चाहिए। एक एल्गोरिदम की दक्षता को सही परिणाम देने के लिए संसाधन के उपयोग के संदर्भ में परिभाषित किया जाता है।
उत्तर।

आम तौर पर किसी Algorithm की एफिशिएंसी को उसकी कम्प्यूटेशनल कॉम्प्लेक्सिटी के हिसाब से देखा जाता है, यानी: Algorithm को एग्जीक्यूट करना कितना मुश्किल है।
किसी Algorithm का एग्जीक्यूशन काफी हद तक इनपुट पर निर्भर करता है। इनपुट कैसा दिखता है, इस आधार पर Algorithm अलग-अलग तरह से काम कर सकता है। उदाहरण के लिए: इंसर्शन सॉर्ट Algorithm की एफिशिएंसी इनपुट ऐरे पर निर्भर करती है। यहां, इंसर्शन सॉर्ट का रनिंग टाइम लीनियर (यानी, $O(n)$) होता है। जबकि रिवर्स ऑर्डर में सॉर्ट किए गए ऐरे के लिए यह क्वाड्रेटिक रनिंग टाइम (यानी, $O(n^2)$) होता है, जिसे सबसे खराब केस कहा जाता है।

Q4. सेट B को उसमें दी गई संबंधित समस्याओं के साथ मैप करें।

एस। एन।	सेट ए	एस.एन.	सेट बी
1	सॉर्टिंग समस्या A		नंबरों की लिस्ट को बढ़ते क्रम में लगाना।
2	ज्यामितिक संकट	B सेट आइटम में कोई आइटम ढूँढना।	
3	ग्राफ समस्या	सी	दो नोड्स के बीच सबसे छोटा रास्ता ढूँढना।
4	न्यूमेरिकल संकट	डी	किसी दिए गए ग्राफ के लिए यूलर ग्राफ ढूँढना।
5	खोज संकट	और	दिए गए लीनियर इक्वेशन के सेट का सॉल्यूशन ढूँढना।
6	स्ट्रिंग प्रसंस्करण	एफ	पॉइंट्स के एक सेट में से उन पॉइंट्स का जोड़ा ढूँढना जिनके बीच सबसे कम दूरी हो।
7		G एक पैराग्राफ में एक शब्द मिलाएं।	

समाधान: सही मिलान इस प्रकार है:

1 – ए, 2 – एफ, 3 – सी, डी, 4 – ई, 5 – बी, 6 – जी।

Q5. प्रॉब्लम सॉल्विंग के डिटरमिनिस्टिक अप्रोच से कब बचना चाहिए? एक उदाहरण देकर समझाएं।

समाधान: एक एल्गोरिदम डिटरमिनिस्टिक होता है अगर इनपुट और प्रोग्राम की स्थिति से अगले आउटपुट का अनुमान लगाया/तय किया जा सके। प्रॉब्लम सॉल्विंग तकनीक का डिटरमिनिस्टिक तरीका उन प्रॉब्लम के लिए सही नहीं है जिनके नतीजे का अंदाज़ा नहीं लगाया जा सकता।

एल्गोरिदम की मूल बातें और
इसके गुण

उदाहरण के लिए, ब्लैकजैक गेम के कार्ड शफलिंग प्रोग्राम का अगला आउटपुट प्लेयर्स को अंदाज़ा नहीं होना चाहिए, भले ही प्रोग्राम का सोर्स कोड दिख रहा हो।

Q6. डायनामिक प्रोग्रामिंग और बैकट्रैकिंग प्रॉब्लम सॉल्विंग अप्रोच में अंतर बताएं। हर टेक्नीक से कौन सी प्रॉब्लम सॉल्व की जा सकती हैं?

सॉल्यूशन: डायनामिक प्रोग्रामिंग एक टेक्नीक है जिसका इस्तेमाल ऑप्टिमाइज़ेशन प्रॉब्लम को सॉल्व करने के लिए बहुत ज़्यादा किया जाता है। ऑप्टिमाइज़ेशन प्रॉब्लम का इस्तेमाल सभी पॉसिबल आउटकम में से मिनिमम या मैक्सिमम रिजल्ट (एकल रिजल्ट) ढूँढने के लिए किया जाता है। बैकट्रैकिंग मेथड में, ब्रूट फ़ोर्स अप्रोच का इस्तेमाल किया जाता है, इसलिए इसका इस्तेमाल ऑप्टिमाइज़ेशन प्रॉब्लम के लिए नहीं किया जाता है। बैकट्रैकिंग अप्रोच उन प्रॉब्लम को सॉल्व करने के लिए सही है जिनके कई रिजल्ट हों और जिनमें से सभी या कुछ एक्सेप्टेबल हों।

बैकट्रैकिंग अप्रोच का इस्तेमाल करके नीचे दी गई समस्याओं को हल किया जा सकता है:

- आठ रानी पहली • मानचित्र रंग • सुडोकू

डायनामिक प्रोग्रामिंग अप्रोच का इस्तेमाल करके नीचे दी गई प्रॉब्लम को सॉल्व किया जा सकता है:

साल

- फ़्लोयड-वारशॉल और बेलमैन जैसे सभी पेयर शॉर्टेस्ट पाथ एल्गोरिदम-फोर्ड •
- 0/1 नैपसेक प्रॉब्लम • चेन मैट्रिक्स
- मल्टीप्लिकेशन
- ट्रैवलिंग सेल्समैन की समस्या

Q7 लालची तकनीक से कौन सी समस्याएं हल की जा सकती हैं?

साल

- फ्रैक्शनल नैप्सैक प्रॉब्लम • मिनिमम कॉस्ट
- स्पैनिंग ट्री • सिंगल सोर्स शॉर्टेस्ट पाथ एल्गोरिदम

Q9 एल्गोरिदम के लिए आम रनिंग टाइम क्या हैं?

उत्तर: कॉन्स्टेंट टाइम, लीनियर टाइम, लॉगरिदमिक टाइम, पॉलीनोमियल टाइम, एक्सपोनेंशियल टाइम और फैक्टोरियल टाइम

Q10 इंडिपेंडेंट सेट प्रॉब्लम को डिफाइन करें।

उत्तर: इंडिपेंडेंट सेट प्रॉब्लम को नोड्स के ऐसे सेट के रूप में डिफाइन किया जा सकता है जो

किसी भी किनारे से जुड़ा हुआ। इस समस्या को बनाने का एक तरीका यह है कि एक स्थिरांक दिया गया है

k और एक ग्राफ G जिसमें n नोड्स (शीर्ष) हैं, एक आकार का एक स्वतंत्र सेट ढूँढें

क।

1.12 आगे की सामग्री

1. एल्गोरिदम का परिचय। थॉमस एच. कॉर्मेन, चार्ल्स ई. लीसरसन, रोनल्ड एल. रिवेस्ट, और विल्फोर्ड स्टीन।
2. एल्गोरिदम डिज़ाइन, जॉन क्लेनबर्ग और ईवा टाडोस, पियर्सन